

Ex 01 Assignment 3

May 2, 2026

Assignment 3 (Rotation Matrices, Euler Angles, and Quaternions, $2 + 2 + 2 = 6$ Points) Author: Rofikul Masud (s82rmasu)

Implement the following functions in Python using NumPy for the representation and conversion of rotation matrices, Euler angles, and quaternions. The Euler-angle convention must be passed explicitly to all functions that convert from or to Euler angles (for example as a string such as XYZ or ZYX).

1. Implement the following functions for rotation matrices: • Conversion quaternion $\rightarrow$ rotation matrix. • Conversion Euler angles $\rightarrow$ rotation matrix.

```
[1]: import numpy as np
      from typing import Union, Tuple
```

Conversion quaternion $\rightarrow$ rotation matrix.

```
[ ]: #(1) Quaternion to Rotation Matrix

class RotationConversionError(Exception):
    pass

def quaternion_to_rotation_matrix(q: np.ndarray) -> np.ndarray:
    #quaternion [w, x, y, z] to 3x3 rotation matrix.

    q = np.asarray(q, dtype=float)

    # shape
    if q.shape != (4,):
        raise RotationConversionError(
            f"Quaternion must have shape (4,), got {q.shape}"
        )

    # Normalize
    norm = np.linalg.norm(q)

    if norm == 0:
        raise RotationConversionError("Quaternion norm cannot be zero.")
```

```

q = q / norm
w, x, y, z = q

# Rotation matrix
R = np.array([
    [1 - 2*(y**2 + z**2), 2*(x*y - z*w), 2*(x*z + y*w)],
    [2*(x*y + z*w), 1 - 2*(x**2 + z**2), 2*(y*z - x*w)],
    [2*(x*z - y*w), 2*(y*z + x*w), 1 - 2*(x**2 + y**2)]
])

return R

```

Test - Quaternion to Rotation Matrix

```

[ ]: # Test 1A: Identity quaternion
q_identity = np.array([1, 0, 0, 0])

R = quaternion_to_rotation_matrix(q_identity)

expected = np.eye(3)

print("Computed R:\n", R)
print("Expected:\n", expected)

print("Identity Test Passed:", np.allclose(R, expected))

# Test 1B: 90° rotation around X-axis
q_x90 = np.array([
    np.cos(np.pi / 4),
    np.sin(np.pi / 4),
    0,
    0
])

R = quaternion_to_rotation_matrix(q_x90)

expected = np.array([
    [1, 0, 0],
    [0, 0, -1],
    [0, 1, 0]
])

print("\nComputed X Rotation:\n", R)
print("Expected X Rotation:\n", expected)

print("90° X Rotation Test Passed:",

```

```
np.allclose(R, expected, atol=1e-6))
```

```
# Test 1C: Orthogonality
```

```
print("Orthogonality Test Passed:",  
      np.allclose(R @ R.T, np.eye(3), atol=1e-6))
```

Computed R:

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

Expected:

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

Identity Test Passed: True

Computed X Rotation:

```
[[ 1.00000000e+00  0.00000000e+00  0.00000000e+00]  
 [ 0.00000000e+00 -2.22044605e-16 -1.00000000e+00]  
 [ 0.00000000e+00  1.00000000e+00 -2.22044605e-16]]
```

Expected X Rotation:

```
[[ 1  0  0]  
 [ 0  0 -1]  
 [ 0  1  0]]
```

90° X Rotation Test Passed: True

Orthogonality Test Passed: True

Euler Angles → Rotation Matrix

```
[7]: def rotation_x(angle):  
      c, s = np.cos(angle), np.sin(angle)  
      return np.array([  
          [1, 0, 0],  
          [0, c, -s],  
          [0, s, c]  
      ])  
  
      def rotation_y(angle):  
          c, s = np.cos(angle), np.sin(angle)  
          return np.array([  
              [c, 0, s],  
              [0, 1, 0],  
              [-s, 0, c]  
          ])  
  
      def rotation_z(angle):
```

```

c, s = np.cos(angle), np.sin(angle)
return np.array([
    [c, -s, 0],
    [s, c, 0],
    [0, 0, 1]
])

def euler_angles_to_rotation_matrix(
    angles: np.ndarray,
    convention: str = "XYZ",
    degrees: bool = False
) -> np.ndarray:
    """
    Passing,

    Args:
        angles: shape (3,)
        convention: "XYZ", "ZYX"
        degrees: if True, converts degrees to radians

    Returns:
        R: shape (3,3)
    """

    angles = np.asarray(angles, dtype=float)

    if angles.shape != (3,):
        raise RotationConversionError(
            f"Angles must have shape (3,), got {angles.shape}"
        )

    convention = convention.upper()

    if len(convention) != 3 or any(axis not in "XYZ" for axis in convention):
        raise RotationConversionError(
            f"Invalid convention '{convention}'"
        )

    if degrees:
        angles = np.deg2rad(angles)

    rotation_map = {
        'X': rotation_x,
        'Y': rotation_y,
        'Z': rotation_z
    }

```

```

R = np.eye(3)

for axis, angle in zip(Convention, angles):
    R = R @ rotation_map[axis](angle)

return R

```

Test

```

[8]: # Test 2A: Zero angles should return identity
angles = np.array([0, 0, 0])

R = euler_angles_to_rotation_matrix(angles, convention="XYZ")

print("Zero Angles Identity Test Passed:",
      np.allclose(R, np.eye(3)))

# Test 2B: 90° around X-axis
angles = np.array([90, 0, 0])

R = euler_angles_to_rotation_matrix(
    angles,
    convention="XYZ",
    degrees=True
)

expected = np.array([
    [1, 0, 0],
    [0, 0, -1],
    [0, 1, 0]
])

print("Euler X Rotation Test Passed:",
      np.allclose(R, expected, atol=1e-6))

# Test 2C: Different conventions produce different results
angles = np.array([45, 30, 60])

R_xyz = euler_angles_to_rotation_matrix(
    angles,
    convention="XYZ",
    degrees=True
)

```

```

R_zyx = euler_angles_to_rotation_matrix(
    angles,
    convention="ZYX",
    degrees=True
)

print("Convention Difference Test Passed:",
      not np.allclose(R_xyz, R_zyx))

# Test 2D: Orthogonality
print("Euler Matrix Orthogonality Test Passed:",
      np.allclose(R_xyz @ R_xyz.T, np.eye(3), atol=1e-6))

```

Zero Angles Identity Test Passed: True
 Euler X Rotation Test Passed: True
 Convention Difference Test Passed: True
 Euler Matrix Orthogonality Test Passed: True
 Round-trip Consistency Check

```

[9]: # Euler -> Matrix
angles = np.array([45, 30, 60])

R_euler = euler_angles_to_rotation_matrix(
    angles,
    convention="XYZ",
    degrees=True
)

# Quaternion for same 45° X only (simple check)
q = np.array([
    np.cos(np.deg2rad(45)/2),
    np.sin(np.deg2rad(45)/2),
    0,
    0
])

R_quat = quaternion_to_rotation_matrix(q)

print("Quaternion Matrix Shape:", R_quat.shape)
print("Euler Matrix Shape:", R_euler.shape)

print("Valid Rotation Matrix from Quaternion:",
      np.allclose(R_quat @ R_quat.T, np.eye(3), atol=1e-6))

```

Quaternion Matrix Shape: (3, 3)
 Euler Matrix Shape: (3, 3)
 Valid Rotation Matrix from Quaternion: True

2 For quaternions please implement the following functions: • Conjugation, • Multiplication, • Norm, • Conversion rotation matrix $\rightarrow$ quaternion. • Conversion Euler angles $\rightarrow$ quaternion.

```
[10]: def quaternion_conjugate(q):
        q = np.asarray(q, dtype=float).reshape(4,)
        w, x, y, z = q
        return np.array([w, -x, -y, -z])

def quaternion_norm(q):
    q = np.asarray(q, dtype=float).reshape(4,)
    return np.linalg.norm(q)

def quaternion_multiply(q1, q2):
    q1 = np.asarray(q1, dtype=float).reshape(4,)
    q2 = np.asarray(q2, dtype=float).reshape(4,)

    w1, x1, y1, z1 = q1
    w2, x2, y2, z2 = q2

    return np.array([
        w1*w2 - x1*x2 - y1*y2 - z1*z2,
        w1*x2 + x1*w2 + y1*z2 - z1*y2,
        w1*y2 - x1*z2 + y1*w2 + z1*x2,
        w1*z2 + x1*y2 - y1*x2 + z1*w2
    ])
```

```
[12]: q = np.array([1, 2, 3, 4])
        expected = np.array([1, -2, -3, -4])

        print("Conjugation Test:",
              np.array_equal(quaternion_conjugate(q), expected))

        q = np.array([1, 2, 2, 2])

        expected = 3.605551275463989

        print("Norm Test:",
              np.isclose(quaternion_norm(q), expected))

        q_identity = np.array([1, 0, 0, 0])
        q = np.array([0.7071, 0.7071, 0, 0])

        result = quaternion_multiply(q_identity, q)

        print("Multiplication Identity Test:",
```

```
np.allclose(result, q))
```

Conjugation Test: True

Norm Test: True

Multiplication Identity Test: True

```
[14]: def rotation_matrix_to_quaternion(R):
    R = np.asarray(R, dtype=float)

    if R.shape != (3, 3):
        raise RotationConversionError("Matrix must be 3x3")

    trace = np.trace(R)

    if trace > 0:
        s = np.sqrt(trace + 1.0) * 2
        w = 0.25 * s
        x = (R[2,1] - R[1,2]) / s
        y = (R[0,2] - R[2,0]) / s
        z = (R[1,0] - R[0,1]) / s

    elif R[0,0] > R[1,1] and R[0,0] > R[2,2]:
        s = np.sqrt(1.0 + R[0,0] - R[1,1] - R[2,2]) * 2
        w = (R[2,1] - R[1,2]) / s
        x = 0.25 * s
        y = (R[0,1] + R[1,0]) / s
        z = (R[0,2] + R[2,0]) / s

    elif R[1,1] > R[2,2]:
        s = np.sqrt(1.0 + R[1,1] - R[0,0] - R[2,2]) * 2
        w = (R[0,2] - R[2,0]) / s
        x = (R[0,1] + R[1,0]) / s
        y = 0.25 * s
        z = (R[1,2] + R[2,1]) / s

    else:
        s = np.sqrt(1.0 + R[2,2] - R[0,0] - R[1,1]) * 2
        w = (R[1,0] - R[0,1]) / s
        x = (R[0,2] + R[2,0]) / s
        y = (R[1,2] + R[2,1]) / s
        z = 0.25 * s

    q = np.array([w, x, y, z])
    return q / np.linalg.norm(q)
```

```
[15]: R = np.eye(3)

q = rotation_matrix_to_quaternion(R)
```



```

expected = np.array([1, 0, 0, 0])

print("Matrix → Quaternion Test:",
      np.allclose(q, expected))

```

Matrix → Quaternion Test: True

```

[16]: def rotation_x(a):
        c, s = np.cos(a), np.sin(a)
        return np.array([[1,0,0],[0,c,-s],[0,s,c]])

    def rotation_y(a):
        c, s = np.cos(a), np.sin(a)
        return np.array([[c,0,s],[0,1,0],[-s,0,c]])

    def rotation_z(a):
        c, s = np.cos(a), np.sin(a)
        return np.array([[c,-s,0],[s,c,0],[0,0,1]])

    def euler_to_quaternion(angles, convention="XYZ", degrees=False):
        angles = np.asarray(angles, dtype=float).reshape(3,)

        if degrees:
            angles = np.deg2rad(angles)

        R = np.eye(3)

        for axis, angle in zip(convention.upper(), angles):
            if axis == "X":
                R = R @ rotation_x(angle)
            elif axis == "Y":
                R = R @ rotation_y(angle)
            elif axis == "Z":
                R = R @ rotation_z(angle)

        return rotation_matrix_to_quaternion(R)

```

```

[17]: angles = np.array([0, 0, 0])

q = euler_to_quaternion(angles)

expected = np.array([1, 0, 0, 0])

print("Euler → Quaternion Identity Test:",
      np.allclose(q, expected))

```

Euler → Quaternion Identity Test: True

```
[18]: class RotationConversionError(Exception):
        pass

    def Rx(a):
        c, s = np.cos(a), np.sin(a)
        return np.array([[1,0,0],[0,c,-s],[0,s,c]])

    def Ry(a):
        c, s = np.cos(a), np.sin(a)
        return np.array([[c,0,s],[0,1,0],[-s,0,c]])

    def Rz(a):
        c, s = np.cos(a), np.sin(a)
        return np.array([[c,-s,0],[s,c,0],[0,0,1]])

[19]: def rotation_matrix_to_euler_angles(R, convention="XYZ", degrees=False):
    R = np.asarray(R, dtype=float)

    if R.shape != (3, 3):
        raise RotationConversionError("Matrix must be 3x3")

    convention = convention.upper()

    if convention != "XYZ":
        raise RotationConversionError("Only XYZ implemented here")

    # handle singularity
    if abs(R[2, 0]) < 1:
        a2 = -np.arcsin(R[2, 0])
        a1 = np.arctan2(R[2,1] / np.cos(a2), R[2,2] / np.cos(a2))
        a3 = np.arctan2(R[1,0] / np.cos(a2), R[0,0] / np.cos(a2))
    else:
        # gimbal lock
        a2 = np.pi/2 if R[2,0] < 0 else -np.pi/2
        a1 = 0
        a3 = np.arctan2(-R[0,1], R[1,1])

    angles = np.array([a1, a2, a3])

    if degrees:
        angles = np.rad2deg(angles)

    return angles
```

```
[20]: R = np.eye(3)

angles = rotation_matrix_to_euler_angles(R)

expected = np.array([0, 0, 0])

print("Test 1 (Identity):",
      np.allclose(angles, expected))
```

Test 1 (Identity): True

```
[21]: R = Rx(np.pi/2)

angles = rotation_matrix_to_euler_angles(R)

print("Test 2 (90 deg X rotation):",
      np.allclose(angles[0], np.pi/2, atol=1e-6))
```

Test 2 (90 deg X rotation): True

```
[ ]: def quaternion_to_rotation_matrix(q):
    q = np.asarray(q, dtype=float).reshape(4,)
    q = q / np.linalg.norm(q)

    w, x, y, z = q

    return np.array([
        [1 - 2*(y**2 + z**2), 2*(x*y - z*w), 2*(x*z + y*w)],
        [2*(x*y + z*w), 1 - 2*(x**2 + z**2), 2*(y*z - x*w)],
        [2*(x*z - y*w), 2*(y*z + x*w), 1 - 2*(x**2 + y**2)]
    ])

def quaternion_to_euler_angles(q, convention="XYZ", degrees=False):
    R = quaternion_to_rotation_matrix(q)

    return rotation_matrix_to_euler_angles(
        R,
        convention=convention,
        degrees=degrees
    )
```

```
[24]: q = np.array([1, 0, 0, 0])

angles = quaternion_to_euler_angles(q)

expected = np.array([0, 0, 0])

print("Test 3 (Quaternion Identity):",
```

```
np.allclose(angles, expected))
```

Test 3 (Quaternion Identity): True

```
[29]: angles = np.array([0.2, 0.4, 0.6])

# Euler → Quaternion
R = Rx(angles[0]) @ Ry(angles[1]) @ Rz(angles[2])
q = rotation_matrix_to_quaternion(R)

# Quaternion → Euler
angles_back = quaternion_to_euler_angles(q)

print("Test 4 (Euler → Q → Euler):",
      np.allclose(angles, angles_back, atol=1e-5))

R = Rx(0.3) @ Ry(0.4) @ Rz(0.5)

q = rotation_matrix_to_quaternion(R)

R_back = quaternion_to_rotation_matrix(q)

print("Test 5 (R → Q → R):",
      np.allclose(R, R_back, atol=1e-6))

angles = np.random.rand(3)

R = Rx(angles[0]) @ Ry(angles[1]) @ Rz(angles[2])

q = rotation_matrix_to_quaternion(R)

angles_back = quaternion_to_euler_angles(q)

print("Test 6 (Random Stability):",
      np.allclose(angles, angles_back, atol=1e-5))
```

Test 4 (Euler → Q → Euler): False

Test 5 (R → Q → R): True

Test 6 (Random Stability): False